Step 2 — The Style Reviewer, Grounded in Your Style Guide With RAG — Solution

> Back to the step · Hint 1 · Hint 2

Story: the Style reviewer, grounded in your **real** style guide through RAG — it searches your document and cites the actual rule, instead of freelancing from general knowledge. **If not:** an ungrounded reviewer looks right until it confidently cites a rule that was never yours.

The style guide (`data/style_guide.md`)

Put at least one **specific, unusual** rule in here — something a model wouldn't invent from general training knowledge. That's what proves retrieval is actually doing the work.

```
# data/style_guide.md

Rule S-3: Every public function must have a one-line docstring.

Rule S-7: Never use single-letter variable names, except loop counters
(i, j, k) and comprehension variables.

Rule S-9: Function names must be snake_case. camelCase is not allowed,
even for functions ported from JavaScript.

Rule S-14: Timeouts must be passed explicitly to every network call; relying
on the library default is a review failure on this team.
```

The RAG layer

```
# src/rag/store.py
import chromadb

COLLECTION_NAME = "style_guide"


def get_collection(embedding_function=None):
    client = chromadb.Client()
```

```
    try:
        client.delete_collection(COLLECTION_NAME)
    except Exception:
        pass
    if embedding_function is None:
        return client.create_collection(COLLECTION_NAME)
    return client.create_collection(
        COLLECTION_NAME, embedding_function=embedding_function
    )
```

```
# src/rag/ingest.py
# Chunk the style guide into individual rules (one per paragraph) and embed
# them, so a diff line can retrieve the specific rules it might violate.
import os

STYLE_GUIDE_PATH = "data/style_guide.md"


def load_rules(path: str = STYLE_GUIDE_PATH) -> list:
    with open(path) as guide_file:
        text = guide_file.read()
    rules = []
    for block in text.split("\n\n"):
        cleaned = block.strip()
        if cleaned.startswith("Rule"):
            rules.append(cleaned)
    return rules


def build_style_store(collection, path: str = STYLE_GUIDE_PATH):
    rules = load_rules(path)
    ids = []
    for index in range(len(rules)):
        ids.append(f"rule-{index}")
    collection.add(ids=ids, documents=rules)
    return collection
```

```
# src/rag/retriever.py
def retrieve_rules(collection, query: str, k: int = 3) -> list:
    found = collection.query(query_texts=[query], n_results=k)
    return found["documents"][0]
```

The Style reviewer

Story: for the changed lines, retrieve the relevant rules and ask the model which lines violate which rule — returning `Finding`s that **cite the retrieved rule text**. The model judges; the rules come from your document, not its memory. **If not:** the reviewer would judge against rules it imagined.

```
# src/prompts/style.txt
You check changed code lines against the team's style rules. ONLY use the
rules provided - never invent a rule. For each clear violation, return a
finding that quotes the rule it breaks. If there are no violations, return an
empty list.

Style rules (the only rules that exist):
{rules}

Changed lines:
{lines}
```

```
# src/agents/style_reviewer.py
# The Style reviewer: retrieve the relevant style rules for the changed lines,
# then ask the model which lines violate which rule - citing the retrieved
# rule, not general knowledge.
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel

from src.rag.retriever import retrieve_rules
from src.schemas import Finding
from src.utils.prompts import load_prompt


class StyleFindings(BaseModel):
    findings: list


class _StyleItem(BaseModel):
    line: str
    rule: str


def run_style(model, collection, files) -> list:
    changed = []
    for one_file in files:
        for line in one_file["added"]:
            changed.append(line)
    if not changed:
        return []
    query = "\n".join(changed)
    rules = retrieve_rules(collection, query)
```

```
    prompt = ChatPromptTemplate.from_template(load_prompt("style"))
    chain = prompt | model.with_structured_output(StyleFindings)
    result = chain.invoke({"rules": "\n".join(rules), "lines": query})
    out = []
    for item in result.findings:
        out.append(Finding(
            severity="low", category="style",
            message=str(item),
        ))
    return out
```

(The model returns a list of violations under `findings`; each is turned into a `Finding`. The `rules` passed into the prompt came from retrieval, so a violation can only cite a rule that's actually in your guide.)

Proving retrieval works

The key test is that the Style reviewer catches a violation of a **specific** rule from your guide. For a changed line like `+def getUserName(id):`, retrieval should pull Rule S-9 (snake_case / no camelCase), and the finding should cite it — a result the model couldn't produce from general knowledge, because "no camelCase even for ported JS" is *your* rule, not a universal one.

Unit-testing retrieval with no model: use a small offline keyword embedding (L2-normalized, like Doc08/Project 3's) so `retrieve_rules(collection, "camelCase function name snake_case")` returns Rule S-9 first — proving the search finds the right rule before any model judges it. The full Style reviewer (with the model) is checked in the integration test.

Tests for this step: the integration test — `run_style` on a `camelCase`-function diff returns a finding that cites the snake_case rule (S-9).